1 . Пошук валідних ходів

Функція `get_valid_moves()` визначає всі можливі позиції для розміщення фігури. Умови валідності:

1. **Однаковий контакт із територією гравця:** Щонайменше одна клітинка фігури (*) повинна торкатися заповненої клітинки гравця на дошці, але не більше однієї.
2. **Не виходить за межі:** Жодна частина фігури не повинна виходити за межі дошки.
3. **Не перекриває супротивника:** Фігура не може займати клітинки, вже заповнені супротивником.

Алгоритм перевіряє всі можливі позиції на дошці для фігури, створюючи набір допустимих координат для подальшого аналізу.

2. Оцінка можливих ходів

Функція `evaluate_move()` обчислює рейтинг кожного валідного ходу, аналізуючи такі критерії:

2.1 Coverage (Покриття)

Цей критерій відповідає за оцінку того, наскільки вигідні комірки покриває фігура. Функція `calculate_heat_map()` генерує "теплову карту" дошки, оцінюючи кожну порожню клітинку. Фактори, які впливають на оцінку:

- **Відстань до центра:** Чим ближче до центра поля, тим вище оцінка клітинки, оскільки центр забезпечує стратегічну перевагу.
- **Близькість до опонента:** Клітинки, що знаходяться поруч із супротивником, отримують додаткові бали. Це дозволяє створювати тиск на опонента.
- **Потенціал клітинки для захоплення:** Якщо клітинка оточує ворожі комірки, вона отримує додаткові бали, що стимулює блокування опонента.

2.2 Encirclement (Оточення)

Цей критерій оцінює, чи створює фігура перешкоди для супротивника.
Функція `calculate_encirclement_score()`:

- Перевіряє, чи оточують клітинки фігури зони розширення опонента.
- Якщо розміщення фігури формує "стіни", які ускладнюють подальше розміщення супротивника, оцінка підвищується.

Додаткові бали нараховуються за блокування стратегічних клітинок опонента.

2.3 Opponent Coverage (Перекриття території опонента)

Цей критерій враховує, наскільки розміщення фігури перекриває клітинки, зайняті супротивником.
Функція `calculate_covered_opponent_territory()` визначає частку клітинок супротивника, які закриваються фігурою. Чим більший цей показник, тим вищий рейтинг ходу.

3. Зважування критеріїв

Для кожного ходу використовується формула:

Copy code
```
загальний_рейтинг = coverage_weight * coverage_score +
                    encirclement_weight * encirclement_score +
                    opponent_coverage_weight * opponent_coverage_score
```

Ваги критеріїв змінюються залежно від прогресу гри:

- На ранніх стадіях гри більша увага приділяється **Coverage**, щоб захопити вільний простір.
- У пізніх стадіях гри важливішим стає **Encirclement**, щоб заблокувати супротивника.

4. Вибір найкращого ходу

Функція `find_best_move()`:

- Обчислює оцінку кожного валідного ходу за формулою.
- Вибирає хід із найвищим рейтингом.

Якщо валідних ходів немає, бот завершує гру.

Особливості реалізації алгоритму

1. **Гнучкість:** Алгоритм працює з дошками будь-якого розміру, підтримуючи фігури довільних форм.
2. **Оптимізація:** Для кожного ходу розраховуються тільки необхідні дані, що зменшує обчислювальну складність.
3. **Блокування опонента:** Особливу увагу приділено створенню перешкод для супротивника, що підвищує шанс на перемогу.
4. **Динамічна стратегія:** Залежно від етапу гри, бот коригує свою поведінку для досягнення оптимальних результатів.

Приклад роботи алгоритму

Початкові умови

Дошка:

```
..................
..................
..................
..0...............
..................
..................
..............X..
..................
..................
```

Фігура:

```
..*
***
```

Результат роботи бота

Бот визначає найкращий хід, наприклад:

```
<got (0): [3, 3]>
```

Дошка після ходу:

```
..................
..................
..................
..000.............
...0..............
..................
..............X..
..................
..................
```